

1. Which list implementation did you use as the snake's body?
 - I choose the LinkedList implementation to use as the snake body.
2. Why did you select that implementation?
 - I chose LinkedList because the snake adds a new head at index 0 every turn. A linked list does this in $O(1)$ by changing pointers, with no shifting and no resizing, while an array has to shift all elements, which takes $O(N)$. The drawback is that removing the tail takes $O(N)$, because my list only has a head pointer and must walk to the end. So the linked list is better for adding the head, but not for the whole step.
3. Which list operations occur during a normal snake movement?
 - During normal move, we call insert at index 0 to add the new head, and call remove at the last index to delete the tail.
4. Which operations occur when the snake eats food?
 - When snake eat the food, we only call insert at index 0 for new head. We do not call remove for the tail, so the snake grow by one segment.
5. What is the worst-case runtime for one movement step?
 - The worst case is $O(N^2)$, where N is the snake length. Inserting the new head is $O(1)$ and removing the tail is $O(N)$. The collision check is the biggest cost: it loops over the body and calls `get(i)` each time, and in a linked list `get(i)` has to walk i nodes. So the loop takes $O(N^2)$ in total. With an ArrayList, `get(i)` is $O(1)$, so the same step would be $O(N)$.
6. How does collision detection affect the runtime of a game step?
 - Collision detection is the slowest part of a step. The game must check the new head against every body segment, so it does $N-1$ comparisons. Because each `get(i)` on a linked list costs $O(i)$, this adds $O(N^2)$ work every turn, and that is larger than the insert and remove costs. It makes the whole step $O(N^2)$ with LinkedList, and only $O(N)$ with ArrayList.
7. What advantages does the array-based implementation provide?
 - The array-based give very fast access to index. Also it does not need to create and delete new node on the heap every single move, so it save some memory overhead.
8. What advantages does the linked implementation provide?
 - The linked list advantage is very fast insert and remove at the front. It only do pointer hookup and not need to shift all the elements or resize the array when capacity is full.
9. Which implementation would you choose if indexed access were the most common operation?

- I will choose ArrayList. Because array provide the fastest time to access any index directly.

10. Which implementation would you choose if insertion and removal at the front were the most common operations?

- I will choose LinkedList. Because insert and remove at the front is $O(1)$ time, and it do not need to shift any other data backward or forward